

HAETAE NTT Verification Artifact

Machine-Checked Functional Correctness for Jasmin

Keywords: HAETAE, Jasmin, EasyCrypt, NTT, Functional Correctness, Montgomery Representation, KPQC

Ji, Yonghyeon

hacker3740@kookmin.ac.kr

Department of Cyber Security
Kookmin University



CSE

**CRYPTO & SECURITY
ENGINEERING Lab**
암호 및 보안 공학 연구실

May 6, 2026

Table of Contents



Artifact Claim

Artifact Map

Mathematical Specifications

Formulation Correspondence

How To Check It

Proof Ideas

Lemma Reasoning

Coverage and Limits

Conclusion



Artifact Claim

Artifact Claim

What the Artifact Proves



The artifact gives a fully machine-checked EasyCrypt proof that the scalar Jasmin NTT code implements the intended HAETAE transform.

Forward NTT

`poly_ntt_jazz` returns an array representing $\text{full_ntt}(p)$, with output bound 2^{24} .

Inverse NTT

`poly_invntt_jazz` returns an array representing `array256_mont(full_invntt( $p$ ))`, the Montgomery-form result expected by HAETA-E arithmetic.

The claim is functional correctness of the scalar NTT primitives, not a full proof of the complete signature scheme.

May 6, 2026



NTTEndToEnd.ec

```

lemma haetae_jasmin_ntt_correct p :
  hoare [Hpoly_extract.M.poly_ntt_jazz :
    NTT_Fq.poly_repr_bound rp p 16 ==>
    NTT_Fq.poly_repr_bound res (NTTFullSpec.full_ntt p) 24].

```

Mathematical formula carried by the code

$$F(p)_i = \sum_{j=0}^{255} p_j \zeta^{(2\text{brev}_8(i)+1)j}, \quad 0 \leq i < 256.$$

$$\forall p \in \mathbb{F}_q^{256}. \quad \{\text{Repr}_{16}(rp, p)\} \text{ poly_ntt_jazz}(rp) \{\text{Repr}_{24}(res, F(p))\}.$$

Thus the EasyCrypt postcondition is not merely termination: it says that the returned Jasmin memory decodes to the full HAETAE forward transform.

Artifact Claim

Forward Theorem as a Formula



Mathematical transform corresponding to `NTTFullSpec.full_ntt`

$$F(p)_i = \sum_{j=0}^{255} p_j \zeta^{(2\text{brev}_8(i)+1)j}, \quad 0 \leq i < 256.$$

Hoare triple proved by `haetae_jasmin_ntt_correct`

$$\{\text{Repr}_{16}(rp, p)\} \text{poly_ntt_jazz}(rp) \{\text{Repr}_{24}(res, F(p))\}.$$

Equivalently, for every input polynomial $p \in \mathbb{F}_q^{256}$ represented by the input memory, the returned Jasmin memory decodes to exactly $F(p)$ and its signed words are bounded by 2^{24} .



Artifact Claim

Inverse NTT Theorem

NTTEndToEnd.ec

```
lemma haetae_jasmin_invntt_correct p :
  hoare [Hpoly_extract.M.poly_invntt_jazz :
    NTT_Fq.poly_repr_bound rp p 16 ==>
    NTT_Fq.poly_repr_bound res
    (NTT_Fq.array256_mont (NTTFullSpec.full_invntt p)) 16].
```

Mathematical formula carried by the code

$$F^{-1}(p)_i = [256]_q^{-1} \sum_{j=0}^{255} p_j \zeta^{-((2\text{brev}_8(j)+1)i)}, \quad \text{Mont}(v)_i = R v_i.$$

$$\forall p \in \mathbb{F}_q^{256}. \quad \{\text{Repr}_{16}(rp, p)\} \text{ poly_invntt_jazz}(rp) \{\text{Repr}_{16}(res, \text{Mont}(F^{-1}(p)))\}.$$

The array256_mont wrapper in the EasyCrypt code is exactly the coefficientwise factor $R = [2^{32}]_q$ in this formula.



Artifact Claim

Inverse Theorem as a Formula

Mathematical transform corresponding to `NTTFullSpec.full_invntt`

$$F^{-1}(p)_i = [256]_q^{-1} \sum_{j=0}^{255} p_j \zeta^{-((2\text{brev}_8(j)+1)i)}, \quad 0 \leq i < 256.$$

$$\text{NTT_Fq.array256_mont}(v)_i = R v_i.$$

Hoare triple proved by `haetae_jasmin_invntt_correct`

$$\{\text{Repr}_{16}(rp, p)\} \text{poly_invntt_jazz}(rp) \{\text{Repr}_{16}(res, R \cdot F^{-1}(p))\}.$$

The factor $R = [2^{32}]_q$ is the Montgomery representation factor appearing in the EasyCrypt postcondition `array256_mont (full_invntt p)`.



Artifact Claim

Why This Was Not Just Reusing Kyber

What could be reused

- ▶ The high-assurance stack: Jasmin source, Jasmin extraction, and EasyCrypt proof.
- ▶ The overall proof shape: refine low-level code to an imperative NTT, then connect the loop to algebra.
- ▶ Standard word-arithmetic and relational Hoare-logic techniques.

What had to be redone

- ▶ HAETAE works with $q = 64513$ and primitive root $\zeta_0 = 426$.
- ▶ The intended transform is the full 256-point spectral NTT over $X^{256} + 1$.
- ▶ The Kyber-style ring-level NTT was not the final HAETAE specification, so a new full spectral specification and algebra bridge were needed.

The artifact closes that HAETAE-specific bridge inside `NTTFullAlgebra.ec`.

The logo consists of two interlocking, stylized hexagonal shapes. The top shape is light green and the bottom shape is light blue. They are composed of multiple parallel lines that create a 3D, layered effect.

Artifact Map



File	Role in the artifact	Checked
NTTEndToEnd.ec	Top-level Hoare theorems for forward and inverse Jasmin entry points.	✓
RefJasminNTT.ec	pRHL refinement from extracted Jasmin code to imperative EasyCrypt procedures.	✓
NTTFullAlgebra.ec	Loop invariants and algebra proving imperative NTT/INTT equal the full spectral specs.	✓
NTTFullSpec.ec	Abstract full NTT and inverse NTT definitions.	✓
NTT_Fq.ec and Fq.ec	Imperative NTT model, coefficient arithmetic, bounds, and array representation predicates.	✓
Montgomery.ec, GFq.ec, Rq.ec	Modular arithmetic, Montgomery reduction, field and ring support.	✓
Array256.ec, BArray1024.ec, WArray1024.ec	Array libraries used by the proof.	✓
Hpoly_extract.ec	Jasmin extraction surface used by the refinement proof.	✓

These files are the checked transitive spine for the final HAETAE NTT functional-correctness theorem.

Artifact Map

Structural Metrics II



File	Lines	Types	Ops	Modules	Lemmas	Hoare/equiv
NTTEndToEnd.ec	104	1	2	0	10	10
NTTFullAlgebra.ec	3085	1	26	0	111	2
NTTFullSpec.ec	160	0	5	0	10	0
NTT_Fq.ec	988	1	21	1	52	0
RefJasminNTT.ec	2609	0	15	0	99	18
Rq.ec	117	2	20	0	0	0
WArray1024.ec	3	0	0	0	0	0

Artifact Map

Structural Metrics III



File	Lines	Types	Ops	Modules	Lemmas	Hoare/equiv
extract/Array256.ec	3	0	0	0	0	0
extract/BArray1024.ec	3	0	0	0	0	0
extract/Hpoly_extract.ec	356	0	0	1	2	2
extract/WArray1024.ec	3	0	0	0	0	0
Total	8846	5	124	3	331	36

The Hoare/equiv column counts deterministic Hoare judgments, probability-one ~~phoare~~ judgments, and relational equiv judgments.

The large proof weight is concentrated in NTTFullAlgebra.ec and RefJasminNTT.ec: algebraic loop reasoning plus Jasmin refinement and word-level bounds.

Artifact Map

Types and Modules as Mathematics



EasyCrypt object	Kind	Mathematical meaning
<code>Rq.poly</code>	type	Carrier $\mathbb{F}_q^{256}$; also read as $\mathbb{F}_q[X]/(X^{256} + 1)$ when multiplication is used.
<code>NTTFullAlgebra.poly</code>	type	Local alias for the same carrier $\mathbb{F}_q^{256}$ inside the stage-invariant proof.
<code>NTTEndToEnd.poly</code>	type	Local alias for the vector quantified in the final Jasmin correctness theorems.
<code>Rq.CR.t</code> , <code>NTT_Fq.CR.t</code>	clone carrier	Ring carrier specialized to <code>coeff</code> ; supplies $0, 1, +, -, \cdot, (\cdot)^{-1}$ over $\mathbb{F}_q$.
<code>Hpoly_extract.M</code>	module	Extracted Jasmin transition system: procedures are state transformers over word arrays.
<code>extract/Hpoly_extract.M</code>	module	Generated extraction-side copy of the same Jasmin transition system, with wrapper equivalence lemmas for public entry points.
<code>NTT_Fq.NTT</code>	module	Imperative mathematical model of forward and inverse NTT loops over field arrays.

Artifact Map

Counted Types and Modules



The five counted type declarations

EasyCrypt type	Set or class represented
<code>Rq.poly</code>	$\mathcal{P} = \mathbb{F}_q^{256}$, also the coefficient class for $\mathbb{F}_q[X]/(X^{256} + 1)$.
<code>NTTFullAlgebra.poly</code>	The same set $\mathcal{P}$, equipped with stage-invariant predicates Stage_ℓ .
<code>NTTEndToEnd.poly</code>	The same set $\mathcal{P}$, used as the quantified input class in the final Hoare contracts.
<code>Rq.CR.t</code>	$\mathbb{F}_q$, instantiated as the commutative-ring carrier for polynomial sums and products.
<code>NTT_Fq.CR.t</code>	$\mathbb{F}_q$, instantiated as the field carrier for exponentiation and twiddle powers.

The three counted modules

EasyCrypt module	State-transforming algorithm
<code>NTT_Fq.NTT</code>	Imperative field-array algorithms $T_{\text{ntt}}, T_{\text{invntt}} : \mathcal{P} \rightarrow \mathcal{P}$ implementing the butterfly loops.
<code>Hpoly_extract.M</code>	Extracted Jasmin machine-state transformer over word arrays; its exported procedures update byte-array memory.
<code>extract/Hpoly_extract.M</code>	Extraction-side wrapper/core transformer; wrapper equivalences relate public entry points to core procedures.

Thus the type count is not five unrelated mathematical universes: it is three presentations of $\mathbb{F}_q^{256}$ and two presentations of $\mathbb{F}_q$, while the module count records three program-transition systems.



Artifact Map

Formula Dictionary for Counted Objects

Base notation used for every correspondence row

Let

$$q = 64513, \quad \mathbb{F} = \mathbb{Z}/q\mathbb{Z}, \quad N = 256, \quad \zeta = \text{incoeff}(426).$$

An EasyCrypt array value of type `coeff Array256.t` is read as a vector $p = (p_0, \dots, p_{255}) \in \mathbb{F}^N$. A Jasmin byte array `rp` is read through

$$D(rp)_i = \text{incoeff}(\text{W32.to_sint}(\text{BArray1024.get32}(rp, i))).$$

Thus the five counted type declarations are carrier declarations, the two modules are transition systems on these carriers, and the ops/lemmas/judgments below are functions, predicates, theorems, and semantic contracts over $\mathbb{F}^N$ and decoded machine states.

Artifact Map

Ops as Formula Schemas I



Ops covered	Mathematical specification
CR.* clone ops in Rq.ec, NTT_Fq.ec GFq.ec, Fq.ec constants and bounds	The ring/field structure on $\mathbb{F}$: $0, 1, +, -, \cdot, (\cdot)^{-1}$, exponentiation, and integer embedding. $q = 64513$; $\text{sint}(x) = \text{asint}(x)$ if $\text{asint}(x) \leq (q-1)/2$ and $\text{asint}(x) - q$ otherwise; $\text{bw32}(w, b) \iff -2^b \leq \text{to_sint}(w) < 2^b$.
compress, decompress	$\text{compress}_d(x) = \lfloor \text{asint}(x)2^d/q + 1/2 \rfloor \bmod 2^d$ and $\text{decompress}_d(y) = \lfloor \lfloor yq/2^d + 1/2 \rfloor \rfloor_q$.
Montgomery.ec reduction ops	$R = 2^k$; $\text{smod}(a, b)$ is the representative congruent to a in $[-b/2, b/2]$; SREDC, REDC, and REDCK specify $T \mapsto TR^{-1} \pmod{q}$ or $T \mapsto TR^{-1} \pmod{N}$ with proved bounds.
Rq.ec ring ops	$0_i = 0, 1_i = \delta_{i0}, (a+b)_i = a_i + b_i, (-a)_i = -a_i$, and $(ab)_i = \sum_{k=0}^{255} a_k b_{i-k}$ with the sign change imposed by $X^{256} = -1$.

Artifact Map

Ops as Formula Schemas II



Ops covered	Mathematical specification
Rq.ntt, Rq.invnnt, basemul	Even/odd split NTT formulas over 128 terms, inverse split formulas with 128^{-1} , and degree-one multiplication modulo $X^2 - \zeta^e$.
NTTFullSpec.ec spectral ops	$F(p)_i = \sum_{j=0}^{255} p_j \zeta^{(2\text{brev8}(i)+1)j}, \quad F^{-1}(p)_i = 256^{-1} \sum_{j=0}^{255} p_j \zeta^{-(2\text{brev8}(j)+1)i}.$
NTTFullAlgebra.ec stage predicates	full_ntt is F , full_invnnt is F^{-1} , and partial_ntt is the corresponding restricted sum. full_stage_spec, fwd_stage_progress, inv_stage_spec, and inv_stage_progress assert that every already-visited butterfly position stores exactly the appropriate partial sum, while unvisited positions still store the previous-stage sum.
RefJasminNTT.ec bound and address ops	The predicates are pointwise: $\text{bound}_{bd}(rp) \iff \forall i \in [0, 256). \quad -2^{bd(i)} \leq \text{to_sint}(rp_i) < 2^{bd(i)}$; address ops are the affine maps selecting the two butterfly cells.
NTTEndToEnd.ec postcondition ops	jasmin_ntt_full_post means $D(rp) = F(p)$ with the proved output bound; jasmin_invnnt_full_post means $D(rp) = R \cdot F^{-1}(p)$ with the inverse output bound.

Artifact Map

Lemma Schemas for All Counted Lemmas



Lemma schema	Mathematician's reading
Evaluation lemmas	Names ending in E, get, or constant names prove concrete equalities such as $q = 64513$, $\zeta^{256} = -1$, and table entries equal powers of ζ .
Range and address lemmas	They prove integer side conditions: indices lie in $[0, 256)$, bit reversal is involutive on this interval, and paired butterfly addresses are distinct.
Reduction and word lemmas	They show machine-word operations denote field arithmetic: $D(a + b) = D(a) + D(b)$, $D(a - b) = D(a) - D(b)$, and $D(\text{fmul}(a, b)) = \text{SREDC}(D(a)D(b))$.
Split and stage lemmas	They prove algebraic recurrence equations for partial sums and then the loop invariant steps $I_k(r, p) \wedge \text{step}(r, r') \Rightarrow I_k(r', p)$, finishing with $I_8(r, p) \Rightarrow r = F(p)$ and the inverse analogue.
Refinement and end-to-end lemmas	They compose decoded-array representation, bound preservation, loop correctness, and final postconditions into $\llbracket \text{Jasmin} \rrbracket(p) = F(p)$ and $\llbracket \text{Jasmin}^{-1} \rrbracket(p) = R \cdot F^{-1}(p)$.

Artifact Map

Ops as Mathematical Functions I



File / op family	Representative ops	Mathematical meaning
Fastexp.ec	fastexp_nbits, fastexp	Recursive exponentiation functions computing x^n .
Fq.ec	q, bw32	Modulus $q = 64513$ and signed-word range predicate $-2^b \leq \text{to_sint}(w) < 2^b$.
GFq.ec	q, as_sint, compress, decompress	Field modulus, signed representative, and rounding maps between coefficients and compressed integers.
Montgomery.ec	R, smod, SREDC, REDC, REDCk	Integer functions specifying Montgomery reduction and signed modular reduction.
Rq.ec	zero, one, &*, &+, &-	Polynomial ring operations in $\mathbb{F}_q[X]/(X^{256} + 1)$.
Rq.ec	zroot, br, ntt, invntt, basemul	Root $\zeta_0 = 426$, bit-reversal, split NTT support, and base multiplication.

Artifact Map

Ops as Mathematical Functions II



File / op family	Representative ops	Mathematical meaning
NTTFullSpec.ec	partial_ntt, full_ntt, full_invntt	Abstract sums defining the HAETAE forward and inverse spectral transforms.
NTT_Fq.ec	R, scale255, array256_mont, array256_mont_inv	Montgomery scaling maps $x \mapsto xR$ and inverse scaling constants.
NTT_Fq.ec	word_to_coeff, poly_repr_bound, barray256_to_poly,	Decoding relation from machine words to $\mathbb{F}_q^{256}$ with signed range bounds.
NTT_Fq.ec	zetas, zetas_inv	Mathematical twiddle tables: field powers of ζ_0 in the loop order used by Jasmin.
NTTFullAlgebra.ec	full_stage_spec, inv_stage_spec, fwd_stage_progress,	Predicates expressing that an array stores all partial NTT or inverse-NTT sums at a stage.
RefJasminNTT.ec	poly_repr_bound_by, stage-bound ops, final_mont_array	Indexed word-bound predicates and the final Montgomery-scaled array during inverse output.
NTTEndToEnd.ec	jasmin_ntt_full_post, jasmin_invntt_full_post	Final postcondition predicates for the two Jasmin entry points.

Artifact Map

Lemma Families as Theorems I



Lemma family	Examples	Mathematical assertion
Exponentiation and constants	<code>fastexpP</code> , <code>qE</code> , <code>scale255E</code>	Computed constants and fast exponentiation equal the intended field-theoretic values.
Montgomery arithmetic	<code>SREDCp_corr</code> , <code>montgomery_reduce_corr_h</code> <code>REDC_corr</code> ,	Machine Montgomery reduction implements the integer function congruent to $TR^{-1} \pmod{q}$.
Word ranges and signed decoding	<code>bw32_weaken</code> , <code>word_to_coeff_*</code> , <code>add_corr</code> , <code>sub_corr</code>	Word operations preserve coefficient meaning and maintain signed bounds.
Specification equivalence	<code>full_ntt_spec_imp</code> , <code>full_invntt_spec_imp</code>	Indexed partial-sum specifications are extensionally equal to the array functions <code>full_ntt</code> and <code>full_invntt</code> .
Root and bit-reversal algebra	<code>exp_zroot_*</code> , <code>bsrev_*</code> , <code>zetas_stage</code>	Powers of ζ_0 and bit-reversed indices normalize to the exponents used in the NTT formulas.

Artifact Map

Lemma Families as Theorems II



Lemma family	Examples	Mathematical assertion
Forward partial-sum split	<code>partial_ntt_split_low,</code> <code>partial_ntt_split_high</code>	A length- 2^{k+1} partial NTT sum splits into two length- 2^k sums combined by a twiddle.
Forward stage induction	<code>forward_nat_pair_refines,</code> <code>fwd_stage_progress_*, ntt_full_ntt</code>	Every forward butterfly preserves the stage invariant; after eight stages the imperative loop equals <code>full_ntt</code> .
Inverse partial-sum split	<code>partial_invntt_split_*,</code> <code>inverse_pair_refines_values</code>	The inverse butterfly merges two inverse partial sums into the next inverse stage.
Inverse stage induction	<code>inv_stage_progress_*, inv_post_exit_full,</code> <code>invntt_full_invntt</code>	The inverse loop plus final scaling equals <code>full_invntt</code> .
Jasmin refinement and bounds	<code>forward_butterfly_*, inverse_butterfly_*,</code> <code>final_mont_array_*</code>	Extracted Jasmin word updates preserve the decoded polynomial and the required per-index bounds.
End-to-end composition	<code>haetae_jasmin_ntt_correct,</code> <code>haetae_jasmin_invntt_correct</code>	Refinement, algebra, and representation lemmas compose into final Jasmin functional-correctness theorems.

Artifact Map

Hoare/equiv Judgments as Semantics



Judgment form

$$\{P\} c \{Q\} \text{ means } \forall s. P(s) \Rightarrow Q(\llbracket c \rrbracket(s)).$$

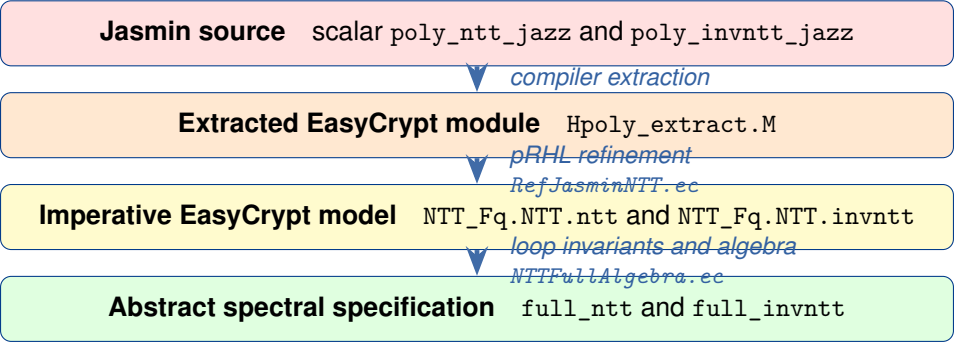
For `phoare`, EasyCrypt asserts the same postcondition with probability 1. Losslessness lemmas discharge termination where needed. An `equiv` judgment is the two-program analogue:

$$P(s_1, s_2) \Rightarrow Q(\llbracket c_1 \rrbracket(s_1), \llbracket c_2 \rrbracket(s_2)).$$

File	Count	Mathematical content
<code>Fq.ec</code>	4	The two Hoare and two probability-one Hoare judgments say <code>__montgomery_reduce</code> computes SREDC and <code>__fqmul</code> computes SREDC(ab).
<code>NTTFullAlgebra.ec</code>	2	Imperative <code>NTT.ntt</code> and <code>NTT.invntt</code> compute <code>full_ntt</code> and <code>full_invntt</code> .
<code>RefJasminNTT.ec</code>	18	Hoare/phoare multiplication obligations and four <code>equiv</code> judgments show extracted Jasmin code refines the imperative model while preserving decoded values and signed-word bounds.
<code>extract/Hpoly_extract.ec</code>	2	Wrapper equivalence judgments relate the public Jasmin wrapper procedures to their extracted core procedures.
<code>NTTEndToEnd.ec</code>	10	The two exported Jasmin entry points satisfy the final pre/post contracts for HAETAE NTT and inverse NTT.
Total	36	All semantic proof obligations in the non-scratch EasyCrypt .ec files counted by the structural scan.

Artifact Map

The Proof Chain



NTTEndToEnd.ec composes these layers into the two final Jasmin correctness theorems.

Artifact Map

The Full HAETAE NTT Specification



Forward transform

$$\text{full_ntt}(p)[i] = \sum_{j=0}^{255} p[j] \cdot \zeta_0^{(2\text{brev}_8(i)+1)j} \pmod{64513}.$$

Evaluation points

The 256 points are:

$$\omega_i = \zeta_0^{2\text{brev}_8(i)+1}.$$

They are the odd powers of a primitive 512th root of unity, arranged by 8-bit reversal.

Inverse transform

The inverse uses the conjugate powers and the factor $1/256$:

$$\text{full_invntt}(y)[i] = 256^{-1} \cdot \sum_j y[j] \omega_j^{-i}.$$

The Jasmin inverse returns this result in Montgomery form.



Mathematical Specifications

Mathematical Specifications

Specification Dictionary



EasyCrypt object	Mathematical object	Role
GFq.cocoeff	Field element in $\mathbb{F}_{64513}$	Coefficient domain
Rq.poly; NTTFullSpec.poly	Vector $(p_0, \dots, p_{255}) \in \mathbb{F}_q^{256}$	Polynomial representation
Rq ring product	Multiplication in $\mathbb{F}_q[X]/(X^{256} + 1)$	Ring context
Rq.zroot / NTT_Fq.zroot	$\zeta_0 = 426 \in \mathbb{F}_q$	Primitive 512th root
Rq.br	$\text{brev}_8(i)$	Bit-reversed index
NTTFullSpec.full_ntt	Full 256-point HAETAE NTT	Forward target spec
NTTFullSpec.full_invntt	Full inverse HAETAE NTT	Inverse target spec
NTT_Fq.NTT.ntt	In-place butterfly algorithm over field arrays	Imperative spec
poly_ntt_jazz	/ Extracted Jasmin entry points in	Machine-code source surface
poly_invntt_jazz	Hpoly_extract.M	

Mathematical Specifications

Forward NTT Target



`NTTFullSpec.full_ntt`

For every input vector $p \in \mathbb{F}_q^{256}$, the specified output $y = \text{full_ntt}(p)$ is:

$$y_i = \sum_{j=0}^{255} p_j \zeta_0^{(2^{\text{brev}_8(i)+1})j} \in \mathbb{F}_q, \quad 0 \leq i < 256.$$

Equivalently, the evaluation point for output index i is:

$$\omega_i = \zeta_0^{2^{\text{brev}_8(i)+1}}, \quad y_i = p(\omega_i).$$

These ω_i are the 256 odd powers of a primitive 512th root, in bit-reversed order.

Mathematical Specifications

Inverse NTT Target



NTTFullSpec.full_invntt

For a spectral vector $y \in \mathbb{F}_q^{256}$, the specified inverse output $p = \text{full_invntt}(y)$ is:

$$p_i = 256^{-1} \sum_{j=0}^{255} y_j \zeta_0^{-((2\text{brev}_8(j)+1)i)} \in \mathbb{F}_q, \quad 0 \leq i < 256.$$

What the Jasmin inverse returns

The end-to-end theorem does not return $\text{full_invntt}(y)$ directly. It returns its Montgomery representation:

$$\text{array256_mont}(\text{full_invntt}(y))_i = \text{full_invntt}(y)_i \cdot R, \quad R = 2^{32} \bmod q.$$

Mathematical Specifications

Partial and Stage Specifications

NTTFullSpec.partial_ntt

$$\text{partial_ntt}(p, \ell, \text{start}, \text{bsj}) = \sum_{s=0}^{\ell-1} \zeta_0^{(2\text{start}+1)\text{brev}_8(s)} p_{\text{brev}_8(\text{bsj} \cdot \ell + s)}.$$

full_ntt_spec r p states:

$$\forall \text{start} \in [0, 256). \quad r_{\text{brev}_8(\text{start})} = \text{partial_ntt}(p, 256, \text{start}, 0).$$

full_ntt_spec_imp proves this indexed statement is exactly the same as the array equation:

$$r = \text{full_ntt}(p).$$

Mathematical Specifications

Imperative Algorithms and Final Hoare Specs



Imperative model

$$\text{NTT.ntt}(p, \text{zetas}) = \text{full_ntt}(p),$$

$$\text{NTT.invntt}(p, \text{zetas_inv}) = \text{full_invntt}(p).$$

These are the theorems `ntt_full_ntt` and `invntt_full_invntt`.

Jasmin entry points

$$\text{poly_ntt_jazz}(rp) \rightsquigarrow \text{full_ntt}(p).$$

$$\text{poly_invntt_jazz}(rp) \rightsquigarrow \text{array256_mont}(\text{full_invntt}(p)).$$

Bounds: 2^{24} for NTT output and 2^{16} for inverse output. Here `rp` represents p by `poly_repr_bound`.

Mathematical Specifications

What Is Not the Final Target



Rq.ntt is not the final HAETAE NTT target

Rq.ec also contains a Kyber-style even/odd split transform:

$$\begin{aligned}
 (\text{Rq.ntt}(p))_{2i} &= \sum_{j=0}^{127} p_{2j} \zeta_0^{(2\text{brev}_8(i)+1)j}, \\
 (\text{Rq.ntt}(p))_{2i+1} &= \sum_{j=0}^{127} p_{2j+1} \zeta_0^{(2\text{brev}_8(i)+1)j}.
 \end{aligned}$$

The artifact explicitly records that this is not the full spectral NTT used as the final HAETAE Jasmin target. NTTFullSpec.ec contains `rq_ntt_is_not_full_ntt_on_one`, a counterexample on the unit polynomial. The end-to-end theorem targets `full_ntt`, not `Rq.ntt`.



Formulation Correspondence

Formulation Correspondence

How To Read This Section



Purpose

This section explains, formula by formula, how the EasyCrypt development turns the Jasmin implementation into a mathematical statement about the HAETAE NTT. The direction is always:

machine array $\longrightarrow$ field vector $\longrightarrow$ partial-sum invariant $\longrightarrow$ full NTT formula.

- ▶ The representation formulas say what mathematical object the machine memory denotes.
- ▶ The butterfly formulas say what one loop body proves.
- ▶ The stage formulas say how all loop bodies compose.
- ▶ The final Hoare triples state the exact theorem about the exported Jasmin entry points.

Complete Declaration Catalog



The presentation folder now contains `haetae-ntt-correspondence-catalog.pdf`. It enumerates every counted EasyCrypt declaration used in the checked functional-correctness chain and records: source line, exact EasyCrypt declaration line, and mathematical correspondence.

kind	type	op	module	lemma	judgment
count	5	124	2	329	34

The structural metrics above expand the syntactic scan to all non-scratch .ec files, adding `extract/Hpoly_extract.ec`'s wrapper module and two wrapper equivalence lemmas.

type	↦	carrier set,
op	↦	total function or predicate,
module	↦	state-transition system,
lemma	↦	checked theorem/schema instance,
hoare/phoare/equiv	↦	program-semantics judgment.

Formulation Correspondence

Catalog Symbol Table



Symbols used uniformly in the full catalog

$q = 64513$, $\mathbb{F}_q = \mathbb{Z}/q\mathbb{Z}$, $N = 256$, $\zeta = [426]_q$, $\text{brev}_8 = 8\text{-bit reversal}$.

For a Jasmin byte array rp ,

$$D(rp)_i = \left[\text{to_sint}(\text{get32}(rp, i)) \right]_q, \quad 0 \leq i < 256.$$

The catalog uses this notation to interpret every EasyCrypt declaration in mathematical form, including all twiddle-table lemmas, all butterfly-stage lemmas, all representation/bound lemmas, and every Hoare-style judgment.



Formulation Correspondence

Representation Relation

Decoding a Jasmin array

For a byte array rp , EasyCrypt decodes the coefficient vector by

$$\llbracket rp \rrbracket_i = \text{incoeff}(\text{W32.to_sint}(\text{BArray1024.get32}(rp, i))), \quad 0 \leq i < 256.$$

This is exactly $\text{NTT_Fq.barray256_to_poly}$.

poly_repr_bound

$$\text{repr}_B(rp, p) \equiv \text{poly_repr_bound}(rp, p, B)$$

$$\iff (\forall i < 256. \llbracket rp \rrbracket_i = p_i) \wedge (\forall i < 256. -2^B \leq \text{to_sint}(\text{get32}(rp, i)) < 2^B).$$

Formulation Correspondence

Representation Relation Explained



What the formula is saying

The Jasmin code manipulates 1024 bytes, grouped as 256 signed 32-bit words. EasyCrypt first defines a decoding map

$$rp \mapsto \llbracket rp \rrbracket \in \mathbb{F}_q^{256}.$$

After this decoding, ordinary mathematical statements can talk about the coefficient vector p rather than about bytes.

Why the bound is part of the specification

$$\text{repr}_B(rp, p) = \text{"}rp \text{ decodes to } p\text{"} \wedge \text{"each signed word is in } [-2^B, 2^B)\text{"}.$$

The equality part is the algebraic content. The bound part is the machine safety content: it lets the proof justify the signed-word arithmetic used by the Jasmin code.

Formulation Correspondence

Forward Butterfly Lemma



Let

$$P_{\ell,s,b}(p) = \sum_{u=0}^{\ell-1} \zeta_0^{(2s+1)\text{brev}_8(u)} p_{\text{brev}_8(b\ell+u)}.$$

For stage k , block ks , offset off , define

$$a = ks2^{8-k} + off, \quad b = a + 2^{7-k},$$

$$z = \text{zetas}[2^k + ks] = \zeta_0^{(2\text{brev}_k(ks)+1)2^{7-k}}.$$

The code assignment

$$t = zr_b, \quad r_b \leftarrow r_a - t, \quad r_a \leftarrow r_a + t$$

is the mathematical update

$$r'_a = r_a + zr_b, \quad r'_b = r_a - zr_b, \quad r'_x = r_x \ (x \notin \{a, b\}).$$

The EasyCrypt lemma `forward_nat_pair_refines` proves:

$$\begin{aligned} r_a &= P_{2^k, \text{brev}_k(ks), 2\text{brev}_{7-k}(off)}(p), \quad r_b = P_{2^k, \text{brev}_k(ks), 2\text{brev}_{7-k}(off)+1}(p) \\ \implies r'_a &= P_{2^{k+1}, \text{brev}_k(ks), \text{brev}_{7-k}(off)}(p), \quad r'_b = P_{2^{k+1}, 2^k+\text{brev}_k(ks), \text{brev}_{7-k}(off)}(p). \end{aligned}$$

Formulation Correspondence

Forward Butterfly Explained



Meaning of $P_{\ell,s,b}$

$P_{\ell,s,b}(p)$ is a partial NTT sum. It looks only at the block numbered b , of length ℓ , and evaluates that block at the exponent pattern determined by s :

$$P_{\ell,s,b}(p) = \sum_{u=0}^{\ell-1} \zeta_0^{(2s+1)\text{brev}_8(u)} p_{\text{brev}_8(b\ell+u)}.$$

Why the butterfly is correct

Before the butterfly, the two touched cells contain two length- 2^k partial sums. Multiplying the second one by

$$z = \zeta_0^{(2\text{brev}_k(ks)+1)2^{7-k}}$$

aligns its exponents with the next stage. The two assignments $r'_a = r_a + zr_b$ and $r'_b = r_a - zr_b$ are therefore exactly the two length- 2^{k+1} partial sums required by the next stage.



Formulation Correspondence

Inverse Butterfly Lemma

Let

$$Q_{\ell,s,b}(y) = \sum_{u=0}^{\ell-1} y_{b\ell+u} \zeta_0^{-((2\text{brev}_8(b\ell+u)+1)s)}.$$

For inverse stage k , block ks , offset off , define

$$\begin{aligned} a &= ks2^{k+1} + off, & b &= a + 2^k, \\ z &= \text{zetas_inv}[256 - 2^{8-k} + ks] \\ &= \zeta_0^{-((2\text{brev}_{7-k}(ks)+1)2^k)}. \end{aligned}$$

The code assignment

$$t = r_a, \quad r_a \leftarrow t + r_b, \quad r_b \leftarrow z(t - r_b)$$

is the mathematical update

$$r'_a = r_a + r_b, \quad r'_b = z(r_a - r_b).$$

`inverse_pair_refines_values` proves:

$$\begin{aligned} r_a &= Q_{2^k, off, 2ks}(y), & r_b &= Q_{2^k, off, 2ks+1}(y) \\ \implies r'_a &= Q_{2^{k+1}, off, ks}(y), & r'_b &= Q_{2^{k+1}, 2^k+off, ks}(y). \end{aligned}$$

Formulation Correspondence

Inverse Butterfly Explained



Meaning of $Q_{\ell,s,b}$

$Q_{\ell,s,b}(y)$ is the inverse counterpart of the forward partial sum. It accumulates spectral coefficients from one block and uses the inverse exponent pattern:

$$Q_{\ell,s,b}(y) = \sum_{u=0}^{\ell-1} y_{b\ell+u} \zeta_0^{-((2\text{brev}_8(b\ell+u)+1)s)}.$$

How the inverse loop differs

The inverse butterfly first forms $r_a + r_b$ and $r_a - r_b$, then multiplies the difference by the inverse-stage root

$$\zeta_0^{-((2\text{brev}_{7-k}(ks)+1)2^k)}.$$

The EasyCrypt proof shows that this merges two length- 2^k inverse partial sums into two length- 2^{k+1} inverse partial sums. The final loop multiplies all coefficients by `zetas_inv[255] =  $256^{-1}$` .

Formulation Correspondence

End-to-End Formula Contracts

Define the two mathematical output vectors

$$\hat{p}_i = \sum_{j=0}^{255} p_j \zeta_0^{(2\text{brev}_8(i)+1)j},$$

$$\check{y}_i = R \cdot 256^{-1} \sum_{j=0}^{255} y_j \zeta_0^{-((2\text{brev}_8(j)+1)i)}, \quad R = 2^{32} \bmod 64513.$$

The final EasyCrypt lemmas are exactly the Hoare triples

$$\{\text{repr}_{16}(rp, p)\} \text{poly_ntt_jazz}(rp) \{\text{repr}_{24}(res, \hat{p})\},$$

$$\{\text{repr}_{16}(rp, y)\} \text{poly_invntt_jazz}(rp) \{\text{repr}_{16}(res, \check{y})\}.$$

The bridge is the pair of checked refinements

$$\begin{aligned} \text{RefJasminNTT} &: \text{Jasmin entry point} \longrightarrow \text{NTT_Fq.NTT}, \\ \text{NTTFullAlgebra} &: \text{NTT_Fq.NTT} \longrightarrow (\hat{p}, \check{y}). \end{aligned}$$



Formulation Correspondence

End-to-End Contracts Explained

Forward contract

The forward theorem says: if the input memory decodes to p with 16-bit signed bounds, then the returned memory decodes to the vector

$$\hat{p}_i = \sum_{j=0}^{255} p_j \zeta_0^{(2\text{brev}_8(i)+1)j},$$

with the larger 24-bit bound needed after additions and multiplications.

Inverse contract

The inverse theorem says the returned memory decodes to the inverse NTT result in Montgomery form:

$$\check{y}_i = R \cdot 256^{-1} \sum_{j=0}^{255} y_j \zeta_0^{-((2\text{brev}_8(j)+1)i)}.$$

The factor R is intentional: HAETAE arithmetic expects the inverse result in Montgomery representation.



How To Check It



How To Check It

Minimal Reproduction

Run location

All commands below are run from:

`haetae-ntt-verify/easycrypt`

Compile the final theorem file

```
easycrypt compile NTTEndToEnd.ec -I .
```

Mathematical judgment checked by the command

$$EC \vdash \forall p \in \mathbb{F}_q^{256}. \{ \text{Repr}_{16}(rp, p) \} \text{ poly_ntt_jazz}(rp) \{ \text{Repr}_{24}(res, F(p)) \},$$

$$EC \vdash \forall p \in \mathbb{F}_q^{256}. \{ \text{Repr}_{16}(rp, p) \} \text{ poly_invntt_jazz}(rp) \{ \text{Repr}_{16}(res, \text{Mont}(F^{-1}(p))) \}.$$

Compiling `NTTEndToEnd.ec` forces EasyCrypt to check the imported proof chain whose conclusion is precisely these two Hoare triples.

How To Check It

Full Artifact Check: Support Libraries



Support files

```
easycrypt compile Array256.ec -I .
easycrypt compile BArray1024.ec -I .
easycrypt compile WArray1024.ec -I .
easycrypt compile Montgomery.ec -I .
easycrypt compile GFq.ec -I .
easycrypt compile Rq.ec -I .
easycrypt compile Fastexp.ec -I .
easycrypt compile Hpoly_extract.ec -I .
easycrypt compile Fq.ec -I .
```

$EC \vdash (\mathbb{F}_{64513}, R, D, \text{Repr}) \text{ with } \text{Repr}_b(rp, p) \Leftrightarrow D(rp) = p \wedge \forall i < 256. -2^b \leq \text{to_sint}(\text{get32}(rp, i)) < 2^b.$

How To Check It

Full Artifact Check: Functional-Correctness Stack



Functional-correctness stack

```
easycrypt compile NTTFullSpec.ec -I .
easycrypt compile NTT_Fq.ec -I .
easycrypt compile RefJasminNTT.ec -I .
easycrypt compile NTTFullAlgebra.ec -I .
easycrypt compile NTTEndToEnd.ec -I .
```

$$EC \vdash (\text{NTT.ntt} = F) \wedge (\text{NTT.invntt} = F^{-1}) \wedge (\text{Hpoly_extract.M} \rightsquigarrow \text{NTT_Fq.NTT}) \wedge \mathcal{H}_{\text{NTT}} \wedge \mathcal{H}_{\text{INTT}}.$$

$$\mathcal{H}_{\text{NTT}} := \{\text{Repr}_{16}(rp, p) \mid \text{poly_ntt_jazz}(rp) \mid \text{Repr}_{24}(res, F(p))\},$$

$$\mathcal{H}_{\text{INTT}} := \{\text{Repr}_{16}(rp, p) \mid \text{poly_invntt_jazz}(rp) \mid \text{Repr}_{16}(res, \text{Mont}(F^{-1}(p)))\}.$$

How To Check It

What Counts as Machine-Checked Here



Checked by EasyCrypt

- ▶ The final Hoare statements in `NTTEndToEnd.ec`.
- ▶ The pRHL refinement lemmas in `RefJasminNTT.ec`.
- ▶ The forward and inverse loop-to-spec theorems in `NTTFullAlgebra.ec`.
- ▶ Array, word, modular arithmetic, and Montgomery support lemmas used by those proofs.

Still part of the trust boundary

- ▶ The EasyCrypt prover, its trusted libraries, and trusted SMT results.
- ▶ The Jasmin compiler theorem connecting extracted code and generated assembly.
- ▶ Foundational assumptions already present in the field, ring, and Montgomery support files.

The artifact claim is therefore: fully machine-checked functional correctness, relative to the standard EasyCrypt/Jasmin trusted base and the explicit foundational assumptions in the support layer.



Proof Ideas

Proof Ideas

The Central Bridge



NTTFullAlgebra.ec

The key new proof obligation is:

$$\text{NTT_Fq.NTT.ntt}(p, \text{zetas}) = \text{full_ntt}(p).$$

The inverse theorem proves the analogous statement for `NTT_Fq.NTT.invntt` and `full_invntt`.

Why this matters

`RefJasminNTT.ec` already connects the extracted Jasmin program to `NTT_Fq.NTT.ntt`. Without the algebra bridge, the artifact would only say that Jasmin matches another program. With the bridge, it says the Jasmin code matches the mathematical HAETAE transform.

Forward NTT Invariant



The iterative butterfly loop is proved by a stage invariant. After stage s , the array equals the partial transform for the first s layers:

$$r = \text{fwd_stage}(s, p).$$

The final stage is $s = 8$, which is the full 256-point transform.

Loop facts

- ▶ The loop indices cover exactly the butterfly pairs for each stage.
- ▶ The twiddle counter is synchronized with the stage and group.
- ▶ Every in-place update preserves the array representation predicate.

Algebra facts

- ▶ The table value $\text{zetas}[k]$ is the right power of ζ_0 .
- ▶ Bit reversal maps loop order to spectral order.
- ▶ The butterfly equations combine two half-sums into the next partial transform.

Proof Ideas

Inverse NTT and Montgomery Form



Inverse loop

The inverse proof follows the reverse butterfly structure. Its invariant tracks how each stage reconstructs the coefficient representation from spectral values. The final scaling by $1/256$ is part of the specification.

Montgomery convention

HAETAE arithmetic stores some values in Montgomery form. The final inverse NTT theorem therefore does not claim:

$$\text{res} = \text{full_invntt}(p).$$

It claims:

$$\text{res} = \text{array256_mont}(\text{full_invntt}(p)).$$

Practical reading: This is not a weakness of the theorem. It is the representation expected by the surrounding HAETAE polynomial arithmetic.



Lemma Reasoning

Proof Dependency Skeleton



Lemma family	Mathematical question	Used by
exp_zroot_*	How do powers of ζ_0 reduce modulo the 512-period?	Exponent normalization
bsrev_*	Which spectral index is selected by each loop counter?	Twiddle and pair lemmas
zetas_stage, zetas_inv_stage	Which root of unity is loaded from the table?	Butterfly steps
fwd_stage_progress_*	Why does one forward butterfly advance one NTT stage?	ntt_full_ntt
inv_stage_progress_* and inv_post_*	Why does the reverse loop compute the inverse spectral formula?	invntt_full_invntt

Each lemma removes one mismatch between program structure and the mathematical definition: loop order, table order, in-place array updates, field exponents, or Montgomery representation.

Lemma Reasoning

Root Period and Evaluation Points



Period facts

$$\zeta_0^{256} = -1, \quad \zeta_0^{512} = 1.$$

`exp_zroot_512` follows by squaring the checked fact $\zeta_0^{256} = -1$. Then `exp_zroot_mod512` lets the proof rewrite:

$$\zeta_0^{e+512n} = \zeta_0^e.$$

Distinct evaluation points The full NTT uses:

$$\omega_i = \zeta_0^{2\text{brev}_8(i)+1}.$$

If $\omega_i = \omega_j$, then:

$$\zeta_0^{2(\text{brev}_8(i) - \text{brev}_8(j))} = 1.$$

Because ζ_0 has period 512, this forces $256 \mid \text{brev}_8(i) - \text{brev}_8(j)$. Both values lie in $[0, 256)$, so $\text{brev}_8(i) = \text{brev}_8(j)$, hence $i = j$.

Bit-Reversal Stage Index



For $0 \leq k < 8$ and $0 \leq ks < 2^k$:

$$\text{brev}_8(2^k + ks) = (2\text{brev}_k(ks) + 1) 2^{7-k}.$$

Bit pattern before reversal	$2^k + ks$ has bit k set to 1, lower k bits equal to ks , and all higher bits zero.
After 8-bit reversal	The set bit moves to position $7 - k$, the lower k bits become $\text{brev}_k(ks)$ above it, and the lower $7 - k$ bits become zero.
Integer value	$2^{7-k} + \text{brev}_k(ks) 2^{8-k} = (2\text{brev}_k(ks) + 1) 2^{7-k}$.

This is the bridge from loop counter $2^k + k_s$ to the spectral exponent used by the mathematical NTT.

Lemma Reasoning

Twiddle Table and Montgomery Cancellation



zetas_stage The table is defined by bit-reversed exponents:

$$\text{zetas}[t] = \zeta_0^{\text{brev}_8(t)}.$$

Substitute $t = 2^k + ks$ and use **bsrev_stage_index**:

$$\text{zetas}[2^k + ks] = \zeta_0^{(2^{\text{brev}_k(ks)+1})2^{7-k}}.$$

Montgomery multiplication The Jasmin table stores the Montgomery value $\zeta R \bmod q$. The reducer satisfies:

$$\text{SREDC}(x) \equiv xR^{-1} \pmod{q}.$$

Therefore the butterfly multiplication computes:

$$\text{SREDC}((\zeta R) a) \equiv \zeta a \pmod{q}.$$

The stored R and the reducer's R^{-1} cancel exactly.



Lemma Reasoning

Forward Butterfly Stage Lemma

The key local step is represented by `fwd_stage_progress_step` and then lifted through loop invariants to `ntt_full_ntt`.

Program update For one pair (a, b) and twiddle w :

$$\begin{aligned} t &= w \cdot b, \\ a' &= a + t, \\ b' &= a - t. \end{aligned}$$

The proof also shows the two array addresses are distinct, so the in-place updates affect only that pair.

Mathematical meaning At stage k , a and b are two partial sums over complementary halves of one residue class. The twiddle from `zetas_stage` supplies exactly the missing factor needed to merge the halves:

$$\text{partial}_k^{\text{low}} + w \text{partial}_k^{\text{high}} = \text{partial}_{k+1}.$$

The lower output uses the same split with a sign change.

Lemma Reasoning

Inverse Loop and Final Scaling



Reverse-stage reasoning

`partial_invntt_split_low` and `partial_invntt_split_high` split the inverse spectral sum into the two halves touched by one reverse butterfly. `zetas_inv_stage` supplies the inverse twiddle factor, so one program pair reconstructs the next inverse partial sum.

The stage lemmas are threaded through the outer invariant until the loop exit postcondition, ending at `inv_outer_exit_post`.

Post-processing The inverse routine has a final scaling pass. The proof uses `zetas_inv_255` and `inv_post_exit_full` to show that this pass multiplies by the required inverse factor:

$$256^{-1}.$$

The end-to-end theorem then wraps the mathematical inverse in `array256_mont`, matching HAETAE's representation convention.

Lemma Reasoning

How EasyCrypt Checks the Reasoning



Mathematical move	EasyCrypt proof form
Equality of polynomials	Array extensionality over all indices in $[0, 256)$.
Loop-to-stage claim	Hoare loop invariant with exact counter, address, and twiddle facts.
Finite-sum rewrite	Big-operator splitting, reindexing, and range side conditions.
Bit-index rewrite	Dedicated <code>bsrev_*</code> lemmas rather than informal bit diagrams.
Modular arithmetic	Field/ring rewriting plus explicit exponent-period lemmas.
Machine-word step	pRHL refinement plus signed-word bounds in <code>RefJasminNTT.ec</code> .

The slides show the mathematical reason; the artifact records each reason as a typed lemma with all range and representation side conditions.



Coverage and Limits

Coverage and Limits

What Is Covered



In scope

- ▶ Scalar Jasmin forward NTT entry point.
- ▶ Scalar Jasmin inverse NTT entry point.
- ▶ Correctness against the full HAETAE spectral NTT specifications.
- ▶ Coefficient representation and bit-width bounds needed by the code.
- ▶ Montgomery reduction behavior needed by the NTT routines.

Out of scope

- ▶ Full HAETAE key generation, signing, and verification.
- ▶ Security proof for the signature scheme.
- ▶ Constant-time or side-channel analysis.
- ▶ AVX2/vectorized implementation correctness.
- ▶ Correctness of unrelated HAETAE arithmetic routines.

Coverage and Limits

Review Path for Artifact Evaluators



1. Start with `NTTEndToEnd.ec`. Read the two final lemmas and the four composition lemmas above them.
2. Open `RefJasminNTT.ec`. Check that the Jasmin entry points refine the imperative NTT and inverse NTT procedures.
3. Open `NTTFullAlgebra.ec`. Inspect `ntt_full_ntt` and `invntt_full_invntt`; these are the loop-to-math bridges.
4. Open `NTTFullSpec.ec`. Confirm that `full_ntt` and `full_invntt` are the intended full HAETAE spectral transforms.
5. Run the compile commands. EasyCrypt must accept the same proof scripts without admitted proof obligations.

Current Status



Layer	Representative theorem or file	Status
Jasmin to imperative model	<code>poly_ntt_core_ref</code> , <code>poly_invntt_core_ref</code>	✓
Imperative forward NTT to full spec	<code>ntt_full_ntt</code>	✓
Imperative inverse NTT to full spec	<code>invntt_full_invntt</code>	✓
Forward Jasmin end-to-end theorem	<code>haetae_jasmin_ntt_correct</code>	✓
Inverse Jasmin end-to-end theorem	<code>haetae_jasmin_invntt_correct</code>	✓

Status change from the older deck: The previous presentation marked the algebra bridge and end-to-end result as partial. The current artifact has those files accepted by EasyCrypt.



Conclusion



Artifact	An EasyCrypt proof stack for the scalar HAETAE Jasmin NTT and inverse NTT.
Result	The final Jasmin entry points are proved functionally correct against the full HAETAE spectral transform specifications.
Evidence	The theorem stack compiles through EasyCrypt, including <code>RefJasminNTT.ec</code> , <code>NTTFullAlgebra.ec</code> , and <code>NTTEndToEnd.ec</code> .
Boundary	This is an NTT functional-correctness artifact, not a whole-signature, side-channel, or AVX2 verification.

Thank You

Questions?

KMU

Ji, Yonghyeon

hacker3740@kookmin.ac.kr

Department of Cyber Security, Kookmin University

Artifact: haetae-ntt-verify/easycrypt

Final file: NTTEndToEnd.ec